# Verilog Assignments

Rakesh Mandal
Roll No. CrS2512

October 31, 2026

## Assignment 1: Full Adder Design

### 1 Problem

Write the Verilog code for a Full Adder that takes three 1-bit inputs `a`, `b` and `carryin`, and gives 1-bit outputs `sum` and `carryout`. Write a testbench that applies all possible input combinations.

### 2 Design Code

```
module full_adder (
    input  a,
    input  b,
    input  carryin,
    output sum,
    output carryout
);

    assign sum      = a ^ b ^ carryin;
    assign carryout = (a & b) | (b & carryin) | (a & carryin);

endmodule
```

### 3 Testbench Code

```
module tb_full_adder;

    reg a, b, carryin;
    wire sum, carryout;
    integer i;

    full_adder uut (
        .a(a),
        .b(b),
        .carryin(carryin),
        .sum(sum),
        .carryout(carryout)
    );

    initial begin
        $display("Full Adder Truth Table");
        $display("-------------------------------");
```

```
18          $display("a  b  carryin | sum  carryout");
19
20          for (i = 0; i < 8; i = i + 1) begin
21              {a, b, carryin} = i;
22              #10;
23
24              $display("%b  %b     %b      |  %b      %b",
25                       a, b, carryin, sum, carryout);
26          end
27
28          $finish;
29      end
30
31  endmodule
```

## 4 Simulation Output

```
Full Adder Truth Table
-------------------------------
a  b  carryin | sum  carryout
0  0     0     |  0      0
0  0     1     |  1      0
0  1     0     |  1      0
0  1     1     |  0      1
1  0     0     |  1      0
1  0     1     |  0      1
1  1     0     |  0      1
1  1     1     |  1      1
```

# Assignment 2: Multiplexer Design

## 1 Problem:

Write the Verilog code for a Multiplexer, that takes in three 1-bit inputs, `datain_0`, `datain_1` and `sel`, and gives `mux_out` 1-bit output.

Write the code for a testbench for the multiplexer and give appropriate inputs to test all possible combinations.

## 2 Verilog Code for the Multiplexer

```
`timescale 1ns/1ps

module multiplexer (
    input datain_0,
    input datain_1,
    input sel,
    output mux_out
);

    assign mux_out = sel ? datain_1 : datain_0;

endmodule
```

## 3 Testbench Code

```
`timescale 1ns/1ps

module tb_multiplexer;

    reg datain_0, datain_1, sel;
    wire mux_out;

    integer i;

    // Instantiate the multiplexer
    multiplexer uut (
        .datain_0(datain_0),
        .datain_1(datain_1),
        .sel(sel),
        .mux_out(mux_out)
    );

    initial begin

        $display("datain_0 datain_1 sel | mux_out");
        $display("------------------------------");

        // Test all possible input combinations
        for (i = 0; i < 8; i = i + 1) begin

            {datain_0, datain_1, sel} = i;

```

```
            #10;

            $display("    %b         %b       %b   |    %b",
                     datain_0, datain_1, sel, mux_out);
        end

        $finish;
    end

endmodule
```

## 4 Simulation Output

```
datain_0 datain_1 sel | mux_out
--------------------------------
    0         0       0   |    0
    0         0       1   |    0
    0         1       0   |    0
    0         1       1   |    1
    1         0       0   |    1
    1         0       1   |    0
    1         1       0   |    1
    1         1       1   |    1
```

# Assignment 3: Decoder Design Using Gate Primitives

## 1 Problem

Decoder Design: design a 2-to-4 decoder using Verilog gate primitives.

## 2 Design Code

```
`timescale 1ns/1ps

module decoder_2to4 (
    input  a,
    input  b,
    output y0,
    output y1,
    output y2,
    output y3
);

    wire a_n, b_n;

    not (a_n, a);
    not (b_n, b);

    and (y0, a_n, b_n);   // a=0, b=0
    and (y1, a_n, b);     // a=0, b=1
    and (y2, a,   b_n);   // a=1, b=0
    and (y3, a,   b);     // a=1, b=1

endmodule
```

## 3 Testbench Code

```
`timescale 1ns/1ps

module tb_decoder_2to4;

    reg a, b;
    wire y0, y1, y2, y3;

    integer i;

    decoder_2to4 uut (
        .a(a),
        .b(b),
        .y0(y0),
        .y1(y1),
        .y2(y2),
        .y3(y3)
    );

    initial begin
        $display("a b | y0 y1 y2 y3");
        $display("------------------");
```

```
22
23          for (i = 0; i < 4; i = i + 1) begin
24              {a, b} = i;
25              #10;
26              $display("%b %b |  %b  %b  %b  %b", a, b, y0, y1, y2, y3);
27          end
28
29          $finish;
30      end
31
32  endmodule
```

## 4 Simulation Output

```
a b | y0 y1 y2 y3
------------------
0 0 |  1  0  0  0
0 1 |  0  1  0  0
1 0 |  0  0  1  0
1 1 |  0  0  0  1
```

# Assignment 4: Decoder Design Using Assign

## 1 Problem

Decoder Design: design a 2-t0-4 decoder using continuous `assign` statements and verify it with a testbench that applies all possible input combinations.

## 2 Design Code

```
`timescale 1ns/1ps

module decoder_2to4_assign (
    input  a,
    input  b,
    output y0,
    output y1,
    output y2,
    output y3
);

    assign y0 = ~a & ~b;    // a=0, b=0
    assign y1 = ~a &  b;    // a=0, b=1
    assign y2 =  a & ~b;    // a=1, b=0
    assign y3 =  a &  b;    // a=1, b=1

endmodule
```

## 3 Testbench Code

```
`timescale 1ns/1ps

module tb_decoder_2to4_assign;

    reg a, b;
    wire y0, y1, y2, y3;

    integer i;

    decoder_2to4_assign uut (
        .a(a),
        .b(b),
        .y0(y0),
        .y1(y1),
        .y2(y2),
        .y3(y3)
    );

    initial begin
        $display("a b | y0 y1 y2 y3");
        $display("------------------");

        for (i = 0; i < 4; i = i + 1) begin
            {a, b} = i;
            #10;
```

```
            $display("%b %b |  %b  %b  %b  %b", a, b, y0, y1, y2, y3);
        end

        $finish;
    end

endmodule
```

## 4 Simulation Output

```
a b | y0 y1 y2 y3
-----------------
0 0 |  1  0  0  0
0 1 |  0  1  0  0
1 0 |  0  0  1  0
1 1 |  0  0  0  1
```

# Assignment 5: Complex Multiplexer Design

## 1 Problem

Write the Verilog code for a Multiplexer, that takes in two inputs, 4 bit `data`, 2 bit `sel`, and gives `mux_out` 1-bit output.

Write the code for a testbench for the multiplexer and give appropriate inputs to test all possible combinations.

Use array concepts.

## 2 Design Code

```
`timescale 1ns/1ps

module multiplexer (
    input [3:0] data,
    input [1:0] sel,
    output mux_out
);

    assign mux_out = data[sel];

endmodule
```

## 3 Testbench Code

```
`timescale 1ns/1ps
module tb_multiplexer;
reg [3:0] data;
reg [1:0] sel;
wire mux_out;
integer i, j;
// Instantiate the multiplexer
multiplexer uut (
.data(data),
.sel(sel),
.mux_out(mux_out)
);
initial begin
$display(" Data Sel | mux_out");
$display("--------------------");
// Test all possible combinations
for (i = 0; i < 16; i = i + 1) begin
for (j = 0; j < 4; j = j + 1) begin
data = i;
sel = j;
#10;
$display(" %b %b | %b",
data, sel, mux_out);
end
end
$finish;
end
```

```
endmodule
```

# 4 Simulation Output

```
 Data Sel | mux_out
--------------------
 0000 00 | 0
 0000 01 | 0
 0000 10 | 0
 0000 11 | 0
 0001 00 | 1
 0001 01 | 0
 0001 10 | 0
 0001 11 | 0
 0010 00 | 0
 0010 01 | 1
 0010 10 | 0
 0010 11 | 0
 0011 00 | 1
 0011 01 | 1
 0011 10 | 0
 0011 11 | 0
 0100 00 | 0
 0100 01 | 0
 0100 10 | 1
 0100 11 | 0
 0101 00 | 1
 0101 01 | 0
 0101 10 | 1
 0101 11 | 0
 0110 00 | 0
 0110 01 | 1
 0110 10 | 1
 0110 11 | 0
 0111 00 | 1
 0111 01 | 1
 0111 10 | 1
 0111 11 | 0
 1000 00 | 0
 1000 01 | 0
 1000 10 | 0
 1000 11 | 1
 1001 00 | 1
 1001 01 | 0
 1001 10 | 0
 1001 11 | 1
 1010 00 | 0
 1010 01 | 1
 1010 10 | 0
 1010 11 | 1
 1011 00 | 1
 1011 01 | 1
 1011 10 | 0
 1011 11 | 1
 1100 00 | 0
 1100 01 | 0
 1100 10 | 1
```

```
1100 11 | 1
1101 00 | 1
1101 01 | 0
1101 10 | 1
1101 11 | 1
1110 00 | 0
1110 01 | 1
1110 10 | 1
1110 11 | 1
1111 00 | 1
1111 01 | 1
1111 10 | 1
1111 11 | 1
```

# Assignment 6: Gray to Binary Converter

## 1 Problem

Design a combinational circuit named "GraytoBin1" for converting a 4 bit gray code to binary number.

Input port of the module takes gray code and output port returns its equivalent binary number.

Implement the circuit with exclusive-OR gates.

Let $b_0, b_1, b_2$ and $b_3$ be the bits representing the binary number, where $b_0$ is the LSB and $b_3$ is the MSB.

Let $g_0, g_1, g_2$ and $g_3$ be the bits representing the gray code of the binary number, where $g_0$ is the LSB and $g_3$ is the MSB.

## 2 Design Code

```
`timescale 1ns/1ps

module GraytoBin1 (
    input  [3:0] gray,    // g3 g2 g1 g0
    output [3:0] bin      // b3 b2 b1 b0
);

    assign bin[3] = gray[3];    // b3 = g3

    xor (bin[2], bin[3], gray[2]);    // b2 = b3 ^ g2
    xor (bin[1], bin[2], gray[1]);    // b1 = b2 ^ g1
    xor (bin[0], bin[1], gray[0]);    // b0 = b1 ^ g0

endmodule
```

## 3 Testbench Code

```
`timescale 1ns/1ps

module tb_GraytoBin1;

    reg  [3:0] gray;
    wire [3:0] bin;

    integer i;

    GraytoBin1 uut (
        .gray(gray),
        .bin(bin)
    );

    initial begin
        $display("Gray | Binary");
        $display("-------------");

        for (i = 0; i < 16; i = i + 1) begin
            gray = i;
```

```
            #10;
            $display("%b | %b", gray, bin);
        end

        $finish;
    end

endmodule
```

## 4 Simulation Output

```
Gray | Binary
-------------
0000 | 0000
0001 | 0001
0010 | 0011
0011 | 0010
0100 | 0111
0101 | 0110
0110 | 0100
0111 | 0101
1000 | 1111
1001 | 1110
1010 | 1100
1011 | 1101
1100 | 1000
1101 | 1001
1110 | 1011
1111 | 1010
```

# Assignment 7: Latch and Flip-flop Design Using Verilog

## 1 Problem

Design the following D flip-flops and a latch in Verilog, and write a testbench:

- Rising edge simple flip-flop
- Rising edge synchronous reset
- Rising edge asynchronous reset, active high
- Rising edge asynchronous reset, active low
- Falling edge simple flip-flop
- Falling edge synchronous reset
- Falling edge asynchronous reset, active high
- Falling edge asynchronous reset, active low
- Latch

## 2 Design Code

```
1  `timescale 1ns/1ps
2  `define DELAY #2 //Flip-flop delay
3
4  // 1. Rising edge, simple
5  module dff_rise_simple (d, clk, q);
6  input d, clk;
7  output q;
8  reg q;
9  always @ (posedge clk) begin
10     q <= `DELAY d;
11 end
12 endmodule
13
14 // 2. Rising edge, synchronous reset (active high)
15 module dff_rise_sync_rst (d, clk, reset, q);
16 input d, clk, reset;
17 output q;
18 reg q;
19 always @ (posedge clk) begin
20     if (reset) q <= 0;
21     else       q <= `DELAY d;
22 end
23 endmodule
24
25 // 3. Rising edge, asynchronous reset, active high (the given example)
26 module dflipflop (d, clk, reset, q);
27 input d, clk, reset;
28 output q;
29 reg q;
30 always @ (posedge clk or posedge reset) begin
31     if (reset) begin
```

```
        q <= 0;
    end
    else begin
        q <= `DELAY d;
    end
end
endmodule

// 4. Rising edge, asynchronous reset, active low
module dff_rise_async_rst_low (d, clk, reset_n, q);
input d, clk, reset_n;
output q;
reg q;
always @ (posedge clk or negedge reset_n) begin
    if (!reset_n) q <= 0;
    else          q <= `DELAY d;
end
endmodule

// 5. Falling edge, simple
module dff_fall_simple (d, clk, q);
input d, clk;
output q;
reg q;
always @ (negedge clk) begin
    q <= `DELAY d;
end
endmodule

// 6. Falling edge, synchronous reset (active high)
module dff_fall_sync_rst (d, clk, reset, q);
input d, clk, reset;
output q;
reg q;
always @ (negedge clk) begin
    if (reset) q <= 0;
    else       q <= `DELAY d;
end
endmodule

// 7. Falling edge, asynchronous reset, active high
module dff_fall_async_rst_high (d, clk, reset, q);
input d, clk, reset;
output q;
reg q;
always @ (negedge clk or posedge reset) begin
    if (reset) q <= 0;
    else       q <= `DELAY d;
end
endmodule

// 8. Falling edge, asynchronous reset, active low
module dff_fall_async_rst_low (d, clk, reset_n, q);
input d, clk, reset_n;
output q;
reg q;
always @ (negedge clk or negedge reset_n) begin
    if (!reset_n) q <= 0;
```

```
    else           q <= `DELAY d;
end
endmodule

// 9. Latch (transparent while en = 1)
module d_latch (d, en, q);
input d, en;
output q;
reg q;
always @ (d or en) begin
    if (en) q <= d;
end
endmodule
```

## 3 Testbench Code

```
\section{Testbench Code}
{\small
\begin{lstlisting}
`timescale 1ns/1ps

module tb_flipflops;

    reg d, clk, reset, reset_n, en;

    wire q_r_simple, q_r_sync, q_r_async_hi, q_r_async_lo;
    wire q_f_simple, q_f_sync, q_f_async_hi, q_f_async_lo;
    wire q_latch;

    // Instantiate all flip-flops and latch

    dff_rise_simple u1 (d, clk, q_r_simple);

    dff_rise_sync_rst u2 (d, clk, reset, q_r_sync);

    dflipflop u3 (d, clk, reset, q_r_async_hi);

    dff_rise_async_rst_low u4 (d, clk, reset_n, q_r_async_lo);

    dff_fall_simple u5 (d, clk, q_f_simple);

    dff_fall_sync_rst u6 (d, clk, reset, q_f_sync);

    dff_fall_async_rst_high u7 (d, clk, reset, q_f_async_hi);

    dff_fall_async_rst_low u8 (d, clk, reset_n, q_f_async_lo);

    d_latch u9 (d, en, q_latch);

    // Clock generation: 10 ns period
    initial clk = 0;
    always #5 clk = ~clk;

    initial begin

        d = 0;
```

```
        reset = 0;
        reset_n = 1;
        en = 0;

        $display("Time | clk d rst rst_n en | R_simple R_sync R_aH R_aL
            | F_simple F_sync F_aH F_aL | Latch");

        $monitor("%4t |  %b   %b  %b    %b   %b |    %b       %b      %b
               %b  |    %b       %b     %b    %b  |   %b",
                 $time, clk, d, reset, reset_n, en,
                 q_r_simple, q_r_sync, q_r_async_hi, q_r_async_lo,
                 q_f_simple, q_f_sync, q_f_async_hi, q_f_async_lo,
                 q_latch);

        // Test data
        #2 d = 1;

        #10 d = 0;
        en = 1;

        #5 d = 1;

        #3 en = 0;

        // Active-high reset test
        #2 reset = 1;

        #10 reset = 0;

        // Active-low reset test
        #5 reset_n = 0;

        #10 reset_n = 1;

        // Latch test
        #5 en = 1;
        d = 0;

        #5 en = 0;
        d = 1;

        #5 en = 1;
        d = 0;

        #5 en = 0;

        #10 $finish;

    end

endmodule
```

# 4 Simulation Output

```
Time | clk d rst rst_n en | R_simple R_sync R_aH R_aL | F_simple F_sync F_aH F_aL | Latch
   0 |  0   0  0    1   0 |    x       x     x    x  |    x       x     x    x  |   x
2000 |  0   1  0    1   0 |    x       x     x    x  |    0       0     0    0  |   x
```

```
5000 |  1   1  0    1   0 |    x       x     x    x  |    0       0     0    0  |   x
7000 |  1   1  0    1   0 |    1       1     1    1  |    0       0     0    0  |   x
10000 |  0   1  0    1   0 |    1       1     1    1  |    0       0     0    0  |   x
12000 |  0   0  0    1   1 |    1       1     1    1  |    1       1     1    1  |   0
15000 |  1   0  0    1   1 |    1       1     1    1  |    1       1     1    1  |   0
17000 |  1   1  0    1   1 |    0       0     0    0  |    1       1     1    1  |   1
20000 |  0   1  0    1   0 |    0       0     0    0  |    1       1     1    1  |   1
22000 |  0   1  1    1   0 |    0       0     0    0  |    1       1     0    1  |   1
25000 |  1   1  1    1   0 |    0       0     0    0  |    1       1     0    1  |   1
27000 |  1   1  1    1   0 |    1       0     0    1  |    1       1     0    1  |   1
30000 |  0   1  1    1   0 |    1       0     0    1  |    1       0     0    1  |   1
32000 |  0   1  0    1   0 |    1       0     0    1  |    1       0     0    1  |   1
35000 |  1   1  0    1   0 |    1       0     0    1  |    1       0     0    1  |   1
37000 |  1   1  0    0   0 |    1       1     1    0  |    1       0     0    0  |   1
40000 |  0   1  0    0   0 |    1       1     1    0  |    1       0     0    0  |   1
42000 |  0   1  0    0   0 |    1       1     1    0  |    1       1     1    0  |   1
45000 |  1   1  0    0   0 |    1       1     1    0  |    1       1     1    0  |   1
47000 |  1   1  0    1   0 |    1       1     1    0  |    1       1     1    0  |   1
50000 |  0   1  0    1   0 |    1       1     1    0  |    1       1     1    0  |   1
52000 |  0   0  0    1   1 |    1       1     1    0  |    1       1     1    1  |   0
55000 |  1   0  0    1   1 |    1       1     1    0  |    1       1     1    1  |   0
57000 |  1   1  0    1   0 |    0       0     0    0  |    1       1     1    1  |   0
60000 |  0   1  0    1   0 |    0       0     0    0  |    1       1     1    1  |   0
62000 |  0   0  0    1   1 |    0       0     0    0  |    1       1     1    1  |   0
65000 |  1   0  0    1   1 |    0       0     0    0  |    1       1     1    1  |   0
67000 |  1   0  0    1   0 |    0       0     0    0  |    1       1     1    1  |   0
70000 |  0   0  0    1   0 |    0       0     0    0  |    1       1     1    1  |   0
72000 |  0   0  0    1   0 |    0       0     0    0  |    0       0     0    0  |   0
75000 |  1   0  0    1   0 |    0       0     0    0  |    0       0     0    0  |   0
testbench.sv:82: $finish called at 77000 (1ps)
```

The time column is in picoseconds (for example, 2000 corresponds to 2 ns). `R_` columns are the rising-edge flip-flops and `F_` columns are the falling-edge flip-flops (`simple` = no reset, `sync` = synchronous reset, `aH`/`aL` = asynchronous reset active high/low).

## References

1. Self-made class notes, Semester 1, Course: Computing Systems 1.
2. Claude